Stakeholder Agent System

Requirements

Regis University - MSSE 692: Software Engineering Practicum I

Matt Bass, Anthony Perez, Joseph Turner, Ramesh Kondala

1-25-2026

Executive Summary 2
User and Personal Analysis 3
User Groups 3
User Goals 3
Usage Patterns 4
User Constraints 4
Synthesis 5
System Context and Scope 6
System Context 6
In Scope 6
Out of Scope 6
Assumptions 7
Constraints 7
Functional Requirements 8
User Story Format 8
Architecturally Significant Requirements 8
Quality Attribute Requirements 12
Performance Scenario 12
Availability / Reliability Scenario 12
Security Scenario 13
Modifiability Scenario 14
Requirements Prioritization (MoSCoW) 16
Requirements Validation Evidence 18
Appendix: Elicitation Evidence 19
Research Summary 19
Elicitation Techniques Used 19
Artifacts 20
Artifact #1: Anthonys' Solo Requirement Elicitation 20
Artifact #2: Joe's Solo Requirement Elicitation 22
Artifact #3: Ramesh's Solo Requirements Elicitation 25
Artifact #4: Matt's solo requirements elicitation 26
Artifact #5: Meeting Notes 32

Executive Summary

Requirements elicitation is a foundational skill in software engineering, yet students frequently lack access to realistic stakeholders with whom to participate in these techniques. In academic environments, stakeholder interaction is often simulated informally or omitted altogether, forcing students to invent requirements rather than discover them through structured inquiry. This gap reduces the effectiveness of requirements engineering education and limits students' preparedness for real-world software projects.

The Stakeholder Agent System is an AI-driven web application designed to address this challenge by providing students with realistic, persistent stakeholder simulation for practicing requirements elicitation. The system enables students to engage in iterative, conversational elicitation with an AI agent that mimics the behavior, priorities, and constraints of a real project stakeholder. By interacting through a chat-based interface, students can practice asking questions, clarifying ambiguities, validating assumptions, and refining requirements over multiple sessions.

The primary users of the system are undergraduate and graduate software engineering students working on academic projects. Secondary users include educators who are interested in the effectiveness of the tool as a learning aid and may review elicitation requirements artifacts for instructional purposes. For the initial minimum viable product (MVP), the system intentionally limits scope to a single project scenario and a single stakeholder persona to ensure timely delivery within a limited academic course timeline.

Key system capabilities include secure user authentication, AI-powered stakeholder interaction, persistence of elicitation history, and reliable session recovery. Critical quality attributes identified for the system include performance (responsive interactions within seconds), availability and reliability (preservation of elicitation data across interruptions), security (strict isolation of user data and compliance with student privacy expectations), and modifiability (the ability to extend scenarios and personas in future iterations).

The Stakeholder Agent System matters because it provides scalable, repeatable, and realistic elicitation practice that reinforces professional requirements engineering behaviors. By shifting students from requirement invention to requirement discovery, the system supports deeper learning outcomes and better aligns academic practice with real-world software engineering expectations. This requirements document establishes a clear and realistic scope for the MVP while defining the functional and quality attribute requirements that will directly inform the architectural decisions in the next phase of the practicum.

User and Personal Analysis

User Groups

Primary User Role: Student

The system is intended to be primarily used by undergraduate and graduate students (possibly advanced secondary) who are either studying requirements and elicitation or applying the requirement elicitation body of knowledge. In later versions of the system, users will be able to either choose from an assortment of pre-defined projects to elicit from or define their own project parameters. The initial minimum viable product (MVP) will simply offer one hard coded project for the students to interact with and elicit requirements from.

Secondary User Role: Educator

Educators have an interest in how useful the tool proves to be in building and reinforcing elicitation skills. They may choose to use the system themselves as students would have to evaluate its efficacy and potentially offer the system to students as part of a course curriculum. Future versions may offer educator reports to track student progress.

Indirect Users: System Administrator, Developer

Developers and System Administrators have an interest in the system's resource utilization and cost to develop, test, maintain, deploy and operate the system.

User Goals

Student

- Opportunity for student to interact with AI to elicit requirements.
- System management of information previously elicited.
- System should simulate student conversation with real world stakeholder.
- Increased student skill in requirement elicitation.
- System should reward good elicitation practices.

Educator

- Evaluation of effectiveness of student elicitation efforts.
- Evaluation of system as a beneficial educational tool.
- Track consistency of student project scope.
- Receive reports on student progress.

Sys Admin / Developer

- Limit resource utilization to develop system (play towards development team skills, open source).
- Automation to limit resource utilization to test system.
- Limit resource utilization to maintain system (plan for extensibility and maintainability).
- Limit resource utilization to deploy system (drain on existing systems or new capital investments).

- Limit resource utilization to operate system (efficient usage of CPU, memory, bandwidth, power etc.).
- Graceful and efficient storage of any needed integration credentials like certificates or keys.
- Efficient cohabitation with other resources hosted in the same IT or cloud system.

Usage Patterns

Student

Students will log into the system with a unique username and password. Future versions may allow student to select between multiple projects or use a configuration page to set up a new project; the initial MVP will offer one hardcoded project scenario to elicit against. Students will primarily interact with the system through a text chat. Future versions may voice or other more advanced interaction, these are outside the scope of the initial MVP. Students may need to perform elicitation work asynchronously over multiple sessions, so the ability to save and review elicitation results is helpful.

Educator

Future versions may provide educators reporting on student elicitation progress. A mechanism to share invitations to create or join an existing "project space" with students could be useful for a somewhat finished version of the software. Educators may choose to generate project details the way students would, to quickly generate details for a concreate assignment in other non-requirement eliciting Software Engineering or Computer Science curricula.

Developers

Require efficient initial development on a limited timeline and ease of revisiting system development in future maintenance and system enhancement scenarios. Need the system to be readily testable.

System Administrators

Need the system to be easy to deploy, update and support.

User Constraints

Student

- System Availability
- Academic Deadlines
- Project Timelines
- Minimum System Requirements
- Internet Access
- Cost

Educator

- Free Time
- Perceived Educational Benefit
- Cost
- Institutional Politics

Developers

- Development Timeline
- Existing Team Skill-Base
- Limited Resource Availability
- Cost
- Stakeholder Interest

Synthesis

The limited timeline and resources available for this project weight developer needs and the core student functionality of being able to elicit requirements from the hardcoded project and store and review elicitation progress (possibly as simple as a chat log) as the most pressing considerations and needed minimal functionality respectively.

To facilitate delivery of a minimum viable product in the limited timeline as well as given the limited existing team skillset and resources, non-critical features will be captured as requirements, marked as outside the current scope, and may be revisited or expounded on in the future.

System Context and Scope

System Context

The system will be hosted by a cloud provider. It will access an LLM API on the backend and use a 3rd party login solution. The user will log in through a front-end web app and conduct elicitation through a chat interface. For the purposes of the initial MVP, a single hard-coded project to elicit from will be provided and a single stakeholder persona will be available to interact with. Students will be able to save their progress, review their elicitation history and resume later.

External Actors

- Student
- Instructor
- API Providers

External Systems

- Authentication System
- Data Storage
- Backend LLM
- Cloud Provider

In Scope

- User Login
- Single Hardcoded Project to Elicit Against
- One Stakeholder Persona
- User Interface (chat)
- Chat History
- Cloud Deployment
- Chrome Support
- AI Guard Rails

Out of Scope

- Reporting
- Automated grading/feedback
- Full lifecycle project management
- External integrations (Worldclass etc.)
- Project selection
- Project configuration
- Stakeholder persona configuration
- Group chats
- Multiple logins to same elicitation history

Assumptions

- Users understand elicitation techniques.
- User has a computer that meets requirements, internet access, and a Chrome browser.
- AI endpoint will be accessible online when needed.
- Chosen cloud provider will make hosted app available when needed.
- Third party authentication will be available for log in when needed.

Constraints

- 8 Week Timeline (3 weeks for implementation)
- Minimal if any Available Capital available
- Team Size (4 developers with other responsibilities)
- Existing Team Skill Set
- Academic use (no billing)
- Potential regulations (FERPA)

Functional Requirements

User Story Format

"As a {Role},

I want {Something},

so that {Desired Outcome}."

Architecturally Significant Requirements

As a Developer,

I want the system to access an LLM API on the backend to generate content for the agent to share,

So that our development cycle can rapidly adopt the advances from the state of the industry in supporting agentic behavior.

As a student,

I want my elicitation chat history to be stored,

So that I can work asynchronously and review my progress when I resume.

As a Developer,

I want data to be stored in a modular 3rd party solution,

So that data is highly available and can be moved to a different modular data store if needed.

As a Developer,

I want data stored for users to be secured from unauthorized access or modification,

So that the user's data is safe and data custody regulations are fulfilled (FERPA, GDPR, etc.).

As an Educator,

I want the Stakeholder Agent to portray people consistent with expectations for stakeholder roles they mimic,

So that the system remains a valuable simulation of real-world requirement elicitation and results of elicitation can be consistent.

As a Developer,

I want to implement a single hardcoded project scenario to elicit from in the MVP,

So that the MVP can deliver value on time in a very limited time window.

As a Developer,

I want to implement a single hardcoded stakeholder persona in the MVP,

So that the MVP can deliver value on time in a very limited time window.

As a student,

I want access to an AI agent that can mimic a project stakeholder in my academic projects for the purposes of requirement elicitation,

So that I can develop my requirement elicitation technique and supplement my project requirements.

As an Educator,

I want the agent to reward good elicitation behavior with higher quality requirement results (in chat),

So that the system provides valuable feedback on elicitation efforts and correctly portrays better technique leading to better elicited results and vice versa.

As an Educator,

I want the stakeholder agent to identify these behaviors as good technique: (demonstrated valuation of stakeholder time, asking permission, demonstrated valuation of stakeholder opinion, does not cut off agent, follow up on details just shared, questions aimed towards stated stakeholder priorities, use of 80/20 rule, demonstration of benefit of specific choices, demonstrate adequate preparation level, demonstrated attention paid to agent responses, ask open ended questions).

So that students learn to implement these established good elicitation behaviors, leading to better elicitation outcomes.

As a Developer,

I want system testing to be automated,

So that releases can be verified then deployed quickly

As a System Administrator,

I want the system to require limited resources to deploy,

So that the cost of deployment is manageable and does not disrupt other normal operations.

As a System Administrator,

I want the system's ongoing resource utilization (CPU cycles, memory, storage, bandwidth, power) to be limited,

So that the cost to run the system is manageable and forecastable.

As a System Administrator,

I want the system to conveniently store needed credentials like certificates or keys,

So that these identifiers are available to facilitate integration with other systems or services when needed.

As a System Administrator,

I want the system to efficiently exist in concert with other systems stored or running on the same IT or cloud systems,

So limited system resources can be effectively shared.

As a student,

I want to log into the system with a username and password,

So that my saved data can be accessed by me and not an unauthorized party.

As a Developer,

I want to integrate a 3rd party authentication service,

So that development time can be sped up, and the authentication can be changed to a different 3rd party solution should that become beneficial.

As a Developer,

I want the user to interact with the agent through a text chat,

So that we can deliver the MVP more quickly within the limited time window.

As a student,

I want the stakeholder agent to be available when I need to perform elicitation work (99% uptime),

So that I can benefit from its offerings when needed, given my erratic schedule.

As a student,

I'd like the AI stakeholder agent to be available as a web app,

So that I can access it from whatever device I have handy (with internet access).

As a student,

I want the cost to buy or license the agent to be low to free,

So that I have access to it given my limited student budget

As a Developer,

I want AI guardrails to be enforced,

So that chat results are constrained to consistent project details, the agent persona is honored and any AI hallucinatory or inappropriate behavior is curbed.

As an Educator,

I want the system to be beneficial for students with a varying range of elicitation experience,

So that more students can benefit from it.

As a student,

I want responses to elicitation inquiries to be returned within 5 seconds,

So that my flow of work is not interrupted, and I do not lose interest.

As a student,

I want the agent system to be intuitive to use,

So that users won't need to spend time to learn how to use the system and potentially lose interest.

As a Developer,

I want API failures to be automatically retried,

So that users won't be frustrated by ambiguous behavior.

As a student,

I want to be presented with clear error messages (when needed),

So that I won't be confused or discouraged by erratic behavior.

As a developer,

I want accessibility features to be included,

So that the app is available to more users and accessibility regulations are met.

As a Developer,

I want a single canned out-of-scope response to questions outside the elicitation scope,

So that we can limit what the MVP will attempt to deliver on an extremely limited development timeline.

As a Developer,

I want to defer implementing accessibility features,

So that we can increase the chances of delivering value on time with the MVP.

Quality Attribute Requirements

Performance Scenario

Under typical conditions with up to 30 concurrent student sessions, the system should provide an AI stakeholder response within 5 seconds for at least 95% of requests.

The user interface will stay smooth and responsive during AI calls, meaning you can continue to navigate, type, and look over previous messages without any interruptions while waiting for a response. This way, everything feels seamless and user-friendly.

Example

Source of Stimulus

30 students participating simultaneously during a course lab session.

Stimulus

Student sends a chat message (average of 500 tokens input and output).

Environment

Operating normally during busy academic hours.

Artifact

AI-driven stakeholder response creation and chat interface

Response

AI responds in less than 5 seconds for over 95% of all requests, ensuring quick and efficient service.

The user interface stays responsive without freezing, and typing or scrolling is always possible during AI calls.

Response Measure

Latency should be 5 seconds or less; UI input lag should be 200 milliseconds or less.

Architectural Impact

Implementing async AI calls, using token-efficient prompts, and cache conversations.

Availability / Reliability Scenario

If the system or AI provider faces a temporary downtime, the application will kindly show a clear error message letting know that the service is momentarily unavailable. Rest assured, your data remains safe and preserved during this time.

Once the system is back up and running, users will be able to pick up right where they left off, continuing their chat in the same project session with all previous conversation history and requirements data preserved. No need to restart the scenario

Source of stimulus

System or external AI provider experiences downtime.

Stimulus

A student sends a chat message while the session is active.

Environment

Experiencing partial service degradation caused by AI API 503 errors and database problems.

The artifact related to the active conversation state and requirements data helps keep things organized and clear.

Response

Show a clear error message: "Service temporarily unavailable. Your chat and requirements are preserved."

Resume Capability

Users can continue their exact session state when the service is restored.

Response Measure

Keep all session data fixed issues in 30 seconds or less; succeed at retry at least 99% of the time.

Architectural Impact

Implements durable session storage, circuit breakers, and exponential backoff retries.

Security Scenario

A user can only see and change their own sessions, conversations, and requirements data. No one else can access this information, except the user themselves and any future authorized instructor views. This helps keep everyone's data secure and private.

All stored data including user profiles, sessions, chat history, and requirements will be kept in a secure data repository with encryption both during transfer and at rest, adhering to institutional or cloud security standards.

Personal or sensitive user information must be safeguarded against unauthorized access and should not appear in logs, error messages, or AI prompts.

Source of Stimulus

Authenticated and unauthenticated users accessing the system

Stimulus

When a user tries to access app features, view data, or circumvent restrictions.

Environment

Normal operation

Artifact

User sessions, chat history, requirements, and user profiles.

Response

Login required

Unauthenticated users are redirected to the login screen.

Data Isolation

Users cannot view/modify other users' sessions or data

Secure Storage

All data is encrypted both during transmission (using TLS 1.3) and when stored.

User Information Secured

No logs exposed, errors, or AI prompts

Response Measure

Ensuring no unauthorized data access and verifying session isolation through penetration testing.

Architectural impact

Session management, row-level database security, input sanitization

Modifiability Scenario

The system will enable addition, update, or deactivation of new stakeholder personas through configuration without requiring code changes. These updates will be reflected in production within a single deployment cycle.

The system will enable users to create and configure new project scenarios—such as domain, constraints, difficulty, and stakeholder traits—without affecting existing scenarios or necessitating database migrations.

The architecture should enable advanced users, such as instructors, to later develop their own custom scenarios and involve stakeholders via a UI, utilizing the same configuration model used for built-in scenarios.

Source of Stimulus

Instructor/admin requests new scenarios or stakeholder behaviors

Stimulus

The admin inputs a stakeholder persona or project scenario through the web interface.

Environment

Production environment with active student sessions.

Artifact

Stakeholder personas, project scenarios, and prompt templates

Response

Adding Stakeholders

A new persona becomes available immediately after saving.

Adding Scenarios

Enables selecting a new project context within a single deployment.

User-created Scenarios

Framework supports building custom future scenarios

Response Measure

Configuration updates are applied within 5 minutes with no downtime for existing sessions.

Architectural Impact

Externalized configuration, versioned personas, and stateless deployments

Requirements Prioritization (MoSCoW)

Priority	Requirement	Rationale	Dependencies	Acceptance Criteria
Must Have	User login and authentication	Core security; protects student data and sessions	None	Users must log in to access chat; unauthenticated users redirected
Must Have	AI chat interface	Core functionality; enables stakeholder interaction	AI provider config	Chat supports multi-turn conversations; responses appear in <5s
Must Have	Single AI stakeholder persona per project	Defines simulation scope; prevents confusion	Chat interface	One consistent persona responds to scenario-relevant questions
Must Have	Project/session model	Organizes conversations and requirements	Data persistence	Sessions can be created, resumed, viewed as distinct entities
Must Have	Data persistence for conversations and requirements	Allow resuming sessions; critical for learning	Database/cloud storage	Chat history and requirements persist across browser closes
Must Have	Cloud deployment	Enables access for students/instructors	All above	App accessible via public URL; handles concurrent users
Should Have	Google/Gmail OAuth login	Improves UX over manual passwords	Login system	Students can “Sign in with Google” using school accounts
Should Have	Configurable AI providers	Flexibility for cost/performance; future-proofs	AI chat	Admin can select OpenAI/ChatGPT/Claude via config
Could Have	Project configuration (team size, duration, etc.)	Adds realism; aligns with proposal objectives	Project model	UI allows selecting project parameters before starting session
Could Have	Export functionality (PDF/MD/CSV)	Supports documentation and grading	Data persistence	Export Requirements button generates downloadable artifact

Could Have	Stakeholder configuration	Enables scenario variety	Stakeholder persona	Admin defines persona traits (tone, strictness, domain)
Could Have	Requirement categories/tags	Improve organization and analysis	Requirements capture	UI supports tagging requirements as Functional/NF, Priority
Could Have	Reporting views	Supports instructor assessment	Data persistence	Dashboard shows session summaries and requirement coverage
Won't Have	Instructor/student management	Out of MVP scope; complex RBAC	N/A	No course rosters or gradebooks in this release
Won't Have	Group chat sessions	Increases state complexity beyond single user	N/A	No multi-student conversations
Won't Have	External LMS integrations	Requires partnerships/APIs beyond 8-week scope	N/A	No Canvas / World Class sync
Won't Have	Full project management	Focus remains on requirements elicitation	N/A	No tasks, sprints, or burndown charts

Requirements Validation Evidence

The team met on Wednesday, January 21, 2025, to review and approve the project requirements. Each member prepared by independently gathering requirements beforehand.

During the meeting, we conducted a walkthrough of everyone's work. Matt presented scenarios for instructor users who could manage student experiences and create custom AI stakeholders. The team decided this was out of scope, choosing instead to focus on the core student-stakeholder interaction. Anthony proposed supporting multiple stakeholders and projects, but we agreed to limit the proof of concept to one stakeholder and one project.

We discussed project scope, identified users and stakeholders, and examined technical constraints. Using MoSCoW and brainstorming, we narrowed down the MVP scope.

All team members approved the requirements at the end of the meeting and agreed on a plan to refine the document. One open issue remains: determining which AI model and API to use, and how to implement a guided chat experience.

We then divided the document work. Joe will compile the executive summary and assemble the final document. Ramesh will handle requirements prioritization using MoSCoW and quality attribute requirements. Matt will develop user personas, system context and scope, and functional requirements. Anthony will provide validation evidence and appendix materials.

Appendix: Elicitation Evidence

Research Summary

We investigated PydanticAI and PydanticAI Graph for building structured AI interactions. Key finding: PydanticAI provides type-safe interfaces for working with language models, which could help ensure consistent stakeholder behavior and validate student inputs against expected formats.

For the front end, we researched TanStack Start as a modern React framework. It offers server-side rendering and type-safe routing, which aligns with our need for a responsive, intuitive web interface.

We explored authentication solutions including WorkOS and Neon Auth to handle secure user login. WorkOS provides enterprise-grade authentication with support for various OAuth providers, while Neon Auth offers a lightweight alternative integrated with Neon's serverless Postgres.

We examined LangGraph for orchestrating AI chat workflows. LangGraph enables building stateful, multi-step conversational agents, which is relevant for managing the complexity of realistic stakeholder interactions that maintain context across a conversation.

We researched AI guardrails to prevent the system from going off-topic or being misused. This includes techniques like prompt engineering, content filtering, and conversation boundary enforcement to ensure students can only interact with the AI for its intended educational purpose.

Through this research, we identified the technical feasibility of creating a guided AI chat experience while also recognizing open questions around API costs at scale and implementing guardrails to prevent off-topic conversations

Elicitation Techniques Used

The team employed several techniques to gather and refine requirements for this document.

We conducted a structured meeting where each member walked through their individually prepared requirements. This walkthrough approach allowed us to review everyone's initial elicitation work and discuss it collectively.

We applied the Five Levels of Requirements framework to ensure comprehensive coverage. Beyond documenting stated requirements, we identified unstated assumptions (such as security and accessibility), unconscious requirements (like AI usage costs and guardrails), and potential conflicts (such as the tension between realistic stakeholder behavior and strict topic control).

The MoSCoW method helped us prioritize features and clarify what the application must include versus what could be deferred. Brainstorming sessions generated ideas about potential issues and edge cases.

Since we lack access to real stakeholders for this project, we simulated stakeholder interactions by role-playing how instructors and students might use the application. This helped us understand user needs and workflows from multiple perspectives

Artifacts

Artifact #1: Anthonys' Solo Requirement Elicitation

Project Overview

What the stakeholder wants:
A web application that allows students to practice gathering requirements from an AI-driven stakeholder without needing access to a real person.

Why:
Students benefit from practicing stakeholder interaction and requirements elicitation in a controlled environment. Finding real stakeholders for student projects is often impractical due to time and financial constraints.

Unstated Requirements

The webpage must be secure and protect user data. AI interaction should be limited to stakeholder conversations, preventing students from using it for unrelated purposes. The website needs to be fast, responsive, and intuitive without requiring extensive setup. Accessibility considerations for disabled users are essential.

The system should support multiple mock projects and stakeholder personalities. The AI must not give away requirements directly when asked, forcing students to use proper elicitation techniques. Students need the ability to save and review their progress throughout the session.

Questions remain about whether the AI should change its mind like a real stakeholder and whether instructors need an admin interface to configure custom scenarios and personas.

Unconscious Requirements

Cost considerations for AI usage at scale must be addressed. AI agent performance and guardrails against bad actors need attention. The system must handle API failures gracefully and retry without losing progress. Helpful error messages should guide users when problems occur.

The consistency of AI responses across sessions needs evaluation to ensure fair assessment.

Potential Issues

The AI must behave realistically while staying strictly on topic, but real stakeholders are inherently unpredictable. The system should be inexpensive yet support unlimited usage, but API costs scale with demand. The stakeholder should provide consistent facts while simulating realistic behaviors like vagueness and changing priorities.

Instructors want deep customization but also minimal setup time. Assessment should be objective and automated, yet requirements elicitation is subjective and resists clean scoring. The application needs full accessibility, mobile support, LMS integration, and broad compatibility while being built quickly and affordably.

The AI difficulty must provide educational value without causing student frustration. Conversation storage for grading conflicts with privacy and minimal retention requirements.

Open Questions

- How do we measure if a student successfully elicited requirements? What scoring criteria should we use?
- What are our data retention and privacy requirements? Do we need to comply with FERPA, GDPR, CCPA, or institutional policies?
- Should instructors be able to audit student sessions?
- Which browsers should we support?

Artifact #2: Joe's Solo Requirement Elicitation

Business Objectives & Vision

Q: What business problem are you trying to solve?

Response:

Students learn requirements elicitation in theory, but they rarely get realistic practice because real stakeholders are not consistently available. This makes it difficult for students to develop confidence and practical skills.

Q: How will you know this solution is successful?

Response:

If students can repeatedly practice elicitation, ask better questions over time, and produce clearer and more complete requirements artifacts, then the system is doing its job.

Stakeholders & Users

Q: Who are the primary users of the system?

Response:

The primary users are graduate-level software engineering students working in project teams as part of their coursework.

Q: Are there secondary stakeholders?

Response:

Yes. Instructors are secondary stakeholders who may review the requirements produced by students, but they are not expected to actively interact with the system during elicitation.

System Scope & Boundaries

Q: What capabilities must the system provide?

Response:

The system must present a realistic stakeholder with a defined business need, respond to student questions in a believable way, and maintain consistency across multiple elicitation sessions.

Q: What should the system explicitly NOT do?

Response:

It should not grade students, manage project schedules, or replace the instructor's role in teaching or evaluation.

Functional Requirements

Q: How do users begin using the system?

Response:

Users should start by providing basic project parameters, such as their experience level, team size, project duration, and general domain, so the stakeholder can be tailored appropriately.

Q: How does elicitation occur?

Response:

Students ask questions iteratively, and the stakeholder responds as a real person would, sometimes providing incomplete information or asking for clarification.

Q: How should previously elicited information be handled?

Response:

The stakeholder should remember what has already been discussed and build on that information instead of repeating or contradicting earlier responses.

Quality Attribute Requirements

Q: What would make the system frustrating or unusable?

Response:

If responses are slow, if the stakeholder forgets prior conversations, or if answers contradict each other, the experience will quickly become frustrating and ineffective.

Q: Are there security or privacy concerns?

Response:

Yes. Each team's project data must be kept separate. One team should never be able to see or infer another team's requirements or discussions.

Data & Information Requirements

Q: What information must be retained by the system?

Response:

The system needs to retain the project configuration, the stakeholder persona, elicited requirements, and any assumptions or constraints that have been identified during discussions.

Constraints & Assumptions

Q: What constraints limit the solution?

Response:

This is an academic project with a fixed timeline, limited development resources, and technical limits related to AI context size.

Q: What assumptions are being made?

Response:

It is assumed that students already understand basic elicitation concepts and will use the system over multiple short sessions rather than in one long interaction.

Risks & Open Issues

Q: What risks could impact the system's effectiveness?

Response:

AI responses could vary too much between sessions, and long projects may exceed the amount of context the system can reasonably retain.

Q: How much ambiguity is pedagogically optimal?

Response:

Some ambiguity is useful because it forces students to ask follow-up questions, but too much ambiguity would make the exercise confusing rather than educational.

Q: Should assumptions be explicitly labeled by the system?

Response:

Labeling assumptions could be helpful for learning, but it should not remove the student's responsibility to identify and validate them.

Q: Should instructors have read-only visibility?

Response:

Read-only access could be useful for evaluation purposes, as long as it does not interfere with the students' elicitation process.

Artifact #3: Ramesh's Solo Requirements Elicitation

What are the main tasks students should be able to accomplish with the tool, such as conducting multiple interviews?

- What details do the student need to provide when setting up a new project, such as team size, skills, duration, domain, etc.?
- How can we best define a stakeholder? Consider their domain, goals, constraints, hidden needs, and purpose to better understand their role.
- What kinds of questions should the AI answer, and which should it refuse as "out of scope" for the stakeholder role?
- What actions should be accessible to instructors, such as creating or editing scenarios, assigning them to students, viewing transcripts, and downloading options?
- Performance expectations such as response time per AI reply?
- What security and privacy measures are necessary? For example, encryption, data retention limits, access controls, compliance requirements, and institutional policies.
- Which browsers and devices should we support, and to what extent? For example, should it be desktop-only or fully responsive on mobile devices?
- What accessibility standards need to be met (e.g., WCAG level, screen reader support, keyboard navigation, magnifying. It should be accessible to all users, including those with physical disabilities. Especially in the USA, any design must be accessible to everyone; otherwise, the team could face legal consequences.
- Error handling of API failures, clear error messages, and automatic retries without losing conversation history.
- What is the estimated scale of usage that the system should comfortably support, such as the number of courses, students per term, and concurrent sessions?
- Monitoring performance tracking, and misuse detection

Artifact #4: Matt's solo requirements elicitation

Stakeholder Agent Elicitation Conversation 1: Student

This piece of requirement definition material imagines a conversation between a developer named (Dev) and a student (Stu) who is being interviewed as a potential user of the stakeholder agent system. The conversation is meant to mimic good organic elicitation of functional requirements. The intention is that this mock conversation can serve as both a simulation that guides the synthesis of functional requirements for the application, and also as a guide to how the system can listen for and act upon certain queues and content in the conversation to affect the level of detail and usefulness of responses using a sentiment score. Questions or comments from Dev that are registered as triggered queues affecting the agent sentiment will be annotated within parenthesis Example: (+5 sentiment) and responses that have their scope increased or decreased by the current sentiment score will similarly be annotated as such Example: (+15% from positive sentiment).
For the purposes of this simulation, we will assume that sentiment starts as mildly hostile (-5, moves to neutral after initial question)

Dev: Good morning, Stu. Thanks for taking the time to talk with me. Like we discussed, I have our chat scheduled for 30 minutes, does that still work for you?
(+5 sentiment, values stakeholder time)

Stu: (+0% from neutral sentiment) Sure. I've experienced the issue with lack of access to real stakeholders for software projects in my Software Engineering curriculum, so after hearing about your project, I am curious to hear what you have in mind, but I am busy and in a hurry.

Dev: Great, is it ok if I ask you some questions that we hope will help us define what the system should do, and how it should work? We think your experience with this space would be helpful in designing a truly helpful AI stakeholder agent.
(+5 sentiment, ask permission)
(+10 sentiment, demonstrated perceived value of interviewee input/expertise)

Stu: (+15% from positive sentiment) Absolutely! This has caused me problems in the past, so being able to share that experience and have it gone into a useful solution sounds great. Last semester, I was working on this project which was an ERP for medium size businesses, and I wish we could have interviewed a seasoned network admin with a lot of experience configuring ERPs like SAP about challenges they have faced and issues they think could emerge when applying the tools and concepts to much smaller businesses. Will your system be able to simulate interactions like that? We couldn't decide how quickly the ML should increase cloud resource availability based upon increased utilization of … [3 more minutes of domain specific details highlighting the value of a domain expert (Network Admin with experience implementing ERP)]

Dev: That's the plan. We think the advent of AI is the solution to how an application can mimic the knowledge and experience of a domain expert. The nuance is in how we make sure all the details the agent creates/shares remains consistent with other project details and free of out of context AI hallucinations.
(+5 sentiment, demonstrate perceived benefit to stakeholder)
(+10 sentiment [accumulated], good use of 80/20 rule)

Stu: (+30% from positive sentiment) Cool, I can see how that could be helpful, and I agree that AI in the backend sounds like a promising way to simulate the experience that's lacking in simulated requirement elicitation in academic software projects like this. I've always felt like I'm spending a good portion of my mental energy coming up with requirements for my academic projects, and disciplined researching ends up giving way to hastily invented requirements based on convenience. I've also found that I seem to tailor requirements to what I already know how to accomplish, which limits the possibilities as well as growth. I can see how that is more helpful with more introductory projects, but as I approach the end of my program, it does feel like there's something missing where it comes to applying the academically studied requirement elicitation practices. Last year we were working on this project where……
[4 minutes of continuing anecdote about instance of previously lacking real-world experience eliciting true requirements]

Dev: Sorry, this text was from the Project's PM. I just wanted to make sure it wasn't something important I needed to include in our conversation.
(-10 sentiment, perceived lack of preparation)
(-10 sentiment, not focused on what interviewee is saying)
(+5 sentiment, interruption is related to elicitation conversation)

Stu: (+15% from decreased positive sentiment [temporarily weighted more negatively]) It happens.

Dev: Ok, now I'd like to hear about your vision about how the program should work and how you think you should interact with it.
(+10 sentiment, perceived valuation of interviewee opinion)
(+5 sentiment, open ended question, allowing interviewee to expound)

Stu: (+30 from positive sentiment) Well, it needs to let me either enter or pick a project topic. It will need to keep track of my project details and existing requirements to ensure any new details or requirements it generates fit with them. Then it needs to add the new requirements to the info it is keeping track of and will reference in the future to ensure cohesion….[anecdote about requirement management in previous project]…It would be really helpful if the agent had a feature to manage my requirements, like prioritize them into a burndown chart and mark them as scheduled, in-progress or implemented. I'd probably like to use a web interface to enter and receive info from the agent. Since we are talking about an AI agent, you should probably be able

to talk to them as well.

Dev: I'd like to hear more about challenges you've had with managing requirements.
(+0 sentiment, continued interest in interviewee experience/opinion, already at max accumulation for this sentiment category)
(+10 sentiment, follow up on details from conversation)

Stu: (+30 from positive sentiment) [discusses previous experience with inconvenient requirement management] (too complex, doesn't match educational needs, steep learning curve, or packaged with other unneeded tools)

Dev: OK, we have been talking for 30 minutes now, which is the time frame we agreed to before-hand. I'd like to respect your time. Is there anything else you'd like to share before we finish?
(+0 sentiment, demonstrated valuation of interviewee time, max sentiment bonus already earned/applied for this cue)

Stu: (+30 from positive sentiment) I can't think of anything specific, but thank you for inviting my participation, and if you'd like to follow up, I'd be open to talking some more, this has been great!

Dev: Thank you for your time and sharing your opinions. We may take you up on your offer to follow up and dive deeper. Might you also be willing to participate in acceptance testing after some development to see if we got some requirements you brought to our attention in our conversation today, right?
(+10 sentiment, demonstrated perceived value of interviewee input/expertise)

Stu: (+40 from positive sentiment) Definitely, it sounds like you guys are on to something here, and if my experience and opinions are so beneficial, I'd be happy to help.

Dev: Great, I'll forward our meeting notes from this conversation by tomorrow in case you'd like to look it over to see if I got anything wrong or if there's anything you think would be useful to add.
(+0 sentiment, demonstrated perceived value of interviewee input/expertise already at maximum)
[next elicitation interview with this stakeholder starts at +40 sentiment]

<u>Stakeholder Agent Elicit Convo 2: Educator</u>
This piece of requirement definition material imagines a conversation between a developer named (Dev) and a Software Engineering or Computer Science Educator/Professor (Paul) who is being interviewed as a potential stakeholder for the stakeholder agent system. The conversation is meant to mimic good organic elicitation of functional requirements. The intention is that this mock conversation can serve as both a simulation that guides the synthesis

of functional requirements for the application, and also as a guide to how the system can listen for and act upon certain queues and content in the conversation to affect the level of detail and usefulness of responses using a sentiment score. Questions or comments from Dev that are registered as triggered queues affecting the agent sentiment will be annotated within parenthesis Example: (+5 sentiment) and responses that have their scope increased or decreased by the current sentiment score will similarly be annotated as such Example: (+15% from positive sentiment).

For the purposes of this simulation, we will assume that sentiment starts as mildly hostile (-5, moves to neutral after initial question).
The technical input from Paul comes partially from a GPT enhanced internet search.

Dev: Good morning, Paul. Thanks for taking the time to talk with me. Like we discussed, I have our chat scheduled for 30 minutes, does that still work for you? And you're happy to answer some questions?
(+5 sentiment, values stakeholder time)
(+5 sentiment, ask permission)

Paul: (+5% from positive sentiment) Yes. This project piques my curiosity since it could benefit my students and solve an educational dilemma that we have been struggling to solve. I'm also curious how you are thinking to solve some of the issues I can think of within the problem domain.

Dev: That's great! We would love to hear about what you think some of the issues will be! As a subject matter expert, your experience and opinions will really be valuable.
(+10 sentiment, demonstrated perceived value of interviewee input/expertise)

Paul: (+15 from positive sentiment) Absolutely. Graduate students come to the Software Engineering program with widely varying degrees of experience eliciting requirements. The ones with previous educational and real-world experience eliciting requirements or probing for business needs generally internalize the theoretical requirement elicitation content well, but career switchers sometimes would benefit from more in-depth elicitation experience. Additionally, many Software Engineering students possess more developed technical skills and less developed soft skill that will benefit requirement elicitation and stakeholder relations in general.
Dev: That's a helpful note / perspective. Thank You! What do you think this system needs to do to make it a useful tool?
(+10 sentiment, demonstrated perceived value of interviewee input/expertise)

Paul: (+25 from positive sentiment) Well, it should keep track of the project details and requirements to be sure they all fit together without ambiguity or conflict. You'll want to use an LLM on the back end as your core intelligence layer to provide the new elicitation conversation dialog in response to inquiries. You should implement some retrieval-augmented generation

(RAG) to customize answers to fit your agent persona and make them feel like they are coming from a real person. Managing your project context or agent memory will be best accomplished with something like LangGraph to manage stateful flows and branching behavior.
Evaluating the quality of the requirement elicitation is of particular interest to me as an educator. If I were to deploy an agent like this for my students to use, it would have even more value to me if it could supply a report on the quality of the requirement elicitation that was performed. This could benefit students as well to see where they elicited well and where they may have opportunities.

Some students may not find requirement elicitation as fun as others, particularly those that are not as skilled at it, yet. Some gamification would probably serve to capture the interest of those students. A separate beginner mode that guides students towards cues the AI is listening for and makes the agent sentiment score visible would probably be helpful for those less skilled or interested students as well. Another thin…

Dev: Wow, that's a wealth of great guidance and info. So much to follow up on. Going back to the evaluation report on the quality of the elicitation. What do you envision there?
(-10 sentiments, cut off interviewee)
(+5 sentiment, follow up on details from conversation)
(+10 sentiment, question towards interviewee stated priority)

(+10 sentiment [accumulated], good use of 80/20 rule)
Paul: (+30% from positive sentiment) I'd like to see what beneficial behaviors the agent was able to detect in the conversation. Things like valuing the agent's time, asking permission, asking open ended questions, etiquette, and use of the 80/20 rule. Stuff like that. I've found that……. [4 min. anecdote about student soft skills and elicitation details]

Dev: Got it! So, by extension, we should be listening for those behaviors in the conversation? Then we use the retrieval-augmented generation to….
(+10 sentiment, follow up on details from conversation)

Paul: (+40% from positive sentiment) Yes exactly, you would use instances of good and poor technique used by the interviewer that were detected by the agent to maintain a sentiment score just like if you were performing data science on social media posts. Then the RAG would use the score to affect the quality of the responses. It will affect the length and usefulness of responses without asking additional follow-up questions etc.

Dev: Amazing! This is exactly the type of information we need to make this agent project a success. We have been talking for 30 minutes now, and I want to respect your time….

(+10 sentiment, demonstrated perceived value of interviewee input/expertise)
(+5 sentiment, values stakeholder time)

Paul: (+55% positive sentiment) Outstanding! Please keep me informed of your progress. I am very interested to see how this turns out.

Dev: Thank you for your time and sharing your opinions. Might you also be willing to participate in acceptance testing after some development to see if we got some requirements you brought to our attention in our conversation today, right? Your expertise would continue to be quite valuable towards success of the project.
(+0 sentiment, demonstrated perceived value of interviewee modifier already at maximum)

Paul (+55% for positive sentiment) I'll try to carve out some time. I'm very busy with [unrelated topic], but I can see you are working on something potentially of great value to the program and our students, so time I can spare will be well spent.

Dev: Great, I'll forward our meeting notes from this conversation by tomorrow in case you'd like to look it over to see if I got anything wrong or if there's anything you think would be useful to add.
(+0 sentiment, demonstrated perceived value of interviewee input/expertise already at maximum)
[next elicitation interview with this stakeholder starts at +55% sentiment]

Artifact #5: Meeting Notes

Huddle notes: 1/21/26 in #all-msse692practicum1Slack AI took notes for this huddle from **7:59:10 PM - 9:21:42 PM EST**. The team met to collaboratively develop their requirements document for a software project focused on AI-assisted requirements elicitation, systematically breaking down each section and assigning responsibilities.

Attendees

@Anthony Perez, @Matt Bass, @Joe Turner, and @R

Summary

- Document Structure and Section Assignment
  - @Matt Bass volunteered to handle user analysis, functional requirements, and system context sections. [1:19:47]
  - @R will manage quality attributes and Moscow requirements sections. [1:12:17]
  - @Joe Turner will compile the final document and write the executive summary. [1:18:12]
  - @Anthony Perez will complete validation evidence, appendix, and provide supporting documentation. [1:20:01]
- Project Scope and User Analysis
  - The team decided to limit the initial project to a single project with one stakeholder to maintain manageable complexity. [50:59]
  - User types initially identified include students and instructors, with potentially identical system interactions. [27:49]
  - The system will focus on AI-assisted requirements elicitation with a chat interface. [39:19]
- Technical Constraints and Considerations
  - The team plans cloud deployment, likely using free student access to Azure, AWS, or Google Cloud. [42:56]
  - Initial authentication will likely use Gmail or similar external services to simplify implementation. [1:07:06]
  - The project is strictly academic, which influences regulatory and integration considerations. [44:53]

Action items

- will create a Slack channel to discuss potential technologies for the project. [1:21:32]
- All team members complete their assigned document sections by Friday midnight. [1:20:21]

- will compile the final document after receiving individual section contributions. [1:18:24]
- Team to research and document challenges in software requirements elicitation for the appendix. [1:14:26]